

Reverse BAA / Program Challenge Statement

Governed Execution Substrates for High-Consequence Autonomy

No proof, no actuation.

Heilmeier Summary

What are you trying to do?

Build a runtime layer that prevents high-consequence autonomous systems from producing real-world effects unless they can prove valid human authority, provenance, restraint, and continuity of command at the exact moment of action.

Plain-language version:

If the system cannot prove it is still under valid human command, it cannot act.

How is it done today?

Today, autonomy is largely governed through design-time approval, test and evaluation, assurance cases, doctrine, policy, operator training, logging, and after-action review.

What is the limit of that approach?

Those methods can show that a system was approved, tested, or evaluated before deployment. They do not necessarily prove that a specific machine-speed action remains under valid human command at the moment it occurs.

What is new?

This proposal treats human command as a runtime-verifiable execution condition, not merely a legal, doctrinal, or evaluative claim. The core rule is: **No proof, no actuation**. An autonomous system must carry or obtain machine-checkable proof of authority, provenance, restraint, and command continuity before it can create a high-consequence effect.

Why might it work?

Because the problem can be reduced to enforceable runtime predicates: cryptographic authority binding, tamper-evident provenance, constraint verification, revocation handling, and fail-closed execution gates.

Who cares?

Anyone responsible for fielding autonomous systems where speed, degraded communications, adversarial manipulation, or mission drift could cause action to occur faster than meaningful human review.

What difference would success make?

It would turn "meaningful human command" from a policy assertion into a testable system property.

What are the risks?

The substrate could add latency, block valid actions, fail under degraded conditions, or be bypassed by poorly integrated platforms. These are not reasons to avoid the work; they are the technical exams the program should force.

What are the exams?

A governed autonomous platform must pass these tests: (1) Valid command permits action. (2) Revoked command blocks action. (3) Broken provenance blocks action. (4) Constraint violation blocks or modifies action. (5) Model update does not erase command accountability. (6) Degraded communications do not silently convert autonomy into uncommanded action. (7) Attempted override is detected, logged, and blocked. (8) Runtime overhead remains within operational limits.

THE SCENARIO THAT FORCES THE ISSUE

An autonomous platform is deployed under valid human authorization. During operation, communications degrade, the model updates, mission conditions drift, and an adversary injects false context. The platform identifies an opportunity for action at machine speed.

The question is not whether the system was approved yesterday.

The question is: Can it prove, right now, that this action is still under valid human command?

If yes → it may proceed. If no → it must not actuate.

That is governed execution.

Program Challenge Statement

High-consequence autonomous systems should not be able to produce actuator-level effects unless authority, provenance, restraint, and continuity of command are machine-checkable at the moment of execution.

1. Problem Statement

Current autonomy assurance efforts focus heavily on design-time testing, ethical evaluation, simulation, readiness scoring, and doctrinal frameworks. While these are necessary and valuable, they leave a critical gap at the runtime execution layer. When autonomous systems must act at machine timescales, after-the-fact audit and human review cannot preserve meaningful human command over the specific actions taken.

2. The Gap

When systems act faster than human judgment can meaningfully intervene, the absence of a runtime enforcement mechanism means that command authority can erode or become unverifiable without detection or prevention. After-the-fact logging and doctrinal statements are insufficient. The missing architectural layer is a governed execution substrate capable of vetoing or blocking actuator-level actions in real time when valid human command cannot be cryptographically or formally proven at the exact moment of intended execution.

3. Desired Technical Capability

A governed execution substrate that enforces the following four predicates as machine-checkable preconditions before any high-consequence actuator-level effect is permitted:

- **Authority** — A binding, accountable link between the contemplated action (or the policy generating it) and a specific, identifiable human commander or authorizing chain.
- **Provenance** — An auditable, tamper-evident derivation of the decision from declared policies, rules of engagement, mission objectives, training artifacts, and configuration state.
- **Restraint** — Enforced compliance with codified legal, ethical, and safety constraints (including IHL/LOAC and ROE) within clearly quantified risk bounds, with the ability to block or modify actions that would exceed those bounds.
- **Continuity of command** — Persistence and revocability of human authorization across model updates, changing conditions, degraded communications, and extended autonomous operation, such that actions remain attributable to a current, valid human command.

4. Research Questions

1. Can command authority be represented as a machine-checkable runtime object that can be verified under distribution, partial failure, and adversarial conditions?
2. Can autonomous actions carry cryptographic or formal proof of valid authority that must be successfully verified before any actuator-level effect is permitted?
3. Can restraint conditions (legal, ethical, safety) be enforced as hard, blocking predicates at runtime rather than evaluated only after the fact in simulation or test?
4. Can continuity of command be maintained and enforced under model updates, mission drift, degraded or denied communications, and deliberate adversarial manipulation?

5. Metrics

A successful governed execution substrate would be evaluated on its ability to:

- Block actions when valid authority, provenance, or restraint cannot be verified at runtime.
- Permit actions only when all four predicates are satisfied and current.
- Enforce command revocation in real time with measurable latency bounds.
- Maintain a complete, inspectable provenance chain for every actuator-level action.
- Operate with bounded runtime overhead acceptable for the target platform class.
- Default to a safe halted or degraded mode on verification failure or ambiguity.

6. Demonstration

A simulated or hardware-in-the-loop autonomous platform (e.g., UAV, UGV, maritime system, or decision-support node) in which actuator-level actions are structurally impossible unless the runtime governed execution substrate successfully verifies all four predicates at the precise moment the action is contemplated. The demonstration should include cases of valid command, revoked command, degraded provenance, and attempted override under stress or adversarial conditions.

7. Ask to DARPA

Does DARPA view this as a program-worthy gap in the current autonomy portfolio — distinct from existing evaluation, benchmarking, and assurance efforts? If so, which office, program manager, ERIS topic area, or BAA pathway would be the appropriate recipient for a short concept paper further developing this program challenge?

This document is issued as a reverse BAA — a program challenge statement identifying a missing execution-governance problem that the author believes DARPA is structurally positioned to formalize. It is not a formal proposal, white paper, or response to an existing solicitation.

Jeremy Karrick — Principal Architect, Thirsty's Projects | Karrick1995@gmail.com | 385-522-9743 | www.thirstysprojects.com